

Dependency and Interaction Analysis for the Animal-to-Human Translation Model

Is the additive logistic regression from the paper justified?

2026-09-02

Contents

1	Introduction	1
2	Data and preprocessing	2
3	Pairwise dependency between predictors	2
3.1	Scatter-plot matrix (pairs plot)	2
3.2	Spearman correlation heatmap (appropriate variables only)	4
3.3	Collinearity check via variance inflation factor (VIF)	5
4	Reproducing the paper's logistic regression	6
5	Random forest and model comparison	7
6	Individual dependency (ICE) plots	8
6.1	What an ICE curve is (in plain terms)	8
6.2	Logistic regression vs. random forest, side by side	8
6.3	Interpreting the ICE plots	11
7	Conclusion: is the additive logistic model justified?	11
8	Suggested response to Reviewer #4 (point 1)	11

1 Introduction

In the paper we investigate if a preclinical drug-disease pair will translate to phase III/IV.

We have fitted an additive logistic regression on six predictors to investigate the association between translation and these predictors.

Reviewer #4 asked whether these predictors are interdependent and whether the additive form hides interactions.

We answer the first point by visualizing and quantifying the pairwise dependency of the predictors. We adress the second point by fitting a random forest that allows for interactions and non-linearities, which can be visualized in individual dependency plots which we will directly compare with the coefficients and dependency plot from the logistic regression model, that assumes linearities and do not allow for interaction. As check if the data require interaction and non-linearity we also compare the test prediction performance of the additive logistic regression an the RF model.

2 Data and preprocessing

```
d <- read_csv("../10_drug_disease_translation_analysis/out/df_translation_associations_NEURO_17.csv", show_col_types = FALSE)

# preprocessing exactly as used for the paper's analysis
d$tested_both_sexes_binary <- factor(d$tested_both_sexes, levels = c(0, 1))
d$prop_blinding <- d$articles_blinding / d$articles_total
d$prop_randomization <- d$articles_randomization / d$articles_total

# outcome: numeric 0/1 (1 = translated) for glm; factor yes/no for the forest
d$target_num <- d$target
d$target_f <- factor(d$target, levels = c(1, 0), labels = c("yes", "no"))

# Standardize continuous predictors used in the regression
vars_to_scale <- c(
  "species_unique",
  "strain_unique",
  "assay_unique",
  "prop_blinding",
  "prop_randomization"
)

d[vars_to_scale] <- scale(d[vars_to_scale])

# nice labels
lab <- c(species_unique = "Species (count)", strain_unique = "Strains (count)",
  assay_unique = "Assays (count)", tested_both_sexes_binary = "Both sexes (0/1)",
  prop_blinding = "Blinding (prop.)", prop_randomization = "Randomization (prop.)")

# "appropriate" variables for rank correlation = ordinal counts + proportions
# (the binary sex variable is NOT a ranked quantity and is handled separately)
appropriate <- c("species_unique", "strain_unique", "assay_unique",
  "prop_blinding", "prop_randomization")

cat("Rows:", nrow(d), "| translated:", sum(d$target_num == 1),
  "| non-translated:", sum(d$target_num == 0), "\n")

## Rows: 3151 | translated: 1839 | non-translated: 1312
```

3 Pairwise dependency between predictors

3.1 Scatter-plot matrix (pairs plot)

We first look at every pair of predictors at once. The diagonal shows each variable's distribution as a **histogram / bar chart**. The lower panels show the joint distribution (counts jittered), and the upper panels the Spearman rank correlation – but only for the ordered/continuous variables. ‘

```
pairs_data <- d %>%
  transmute(`Species` = species_unique, `Strains` = strain_unique,
    `Assays` = assay_unique, `Both sexes` = tested_both_sexes_binary,
    `Blinding` = prop_blinding, `Randomization` = prop_randomization)

p_pairs <- ggpairs(
  pairs_data,
```

```

upper = list(continuous = wrap("cor", method = "spearman", size = 3)),
lower = list(continuous = wrap("points", alpha = 0.15, size = 0.4,
                             position = position_jitter(width = 0.2, height = 0)),
            combo = wrap("box_no_facet", outlier.size = 0.4)),
diag = list(continuous = wrap("barDiag", bins = 15)) # histogram, not density
) + theme_bw(base_size = 9)

```

p_pairs

visualize first ...

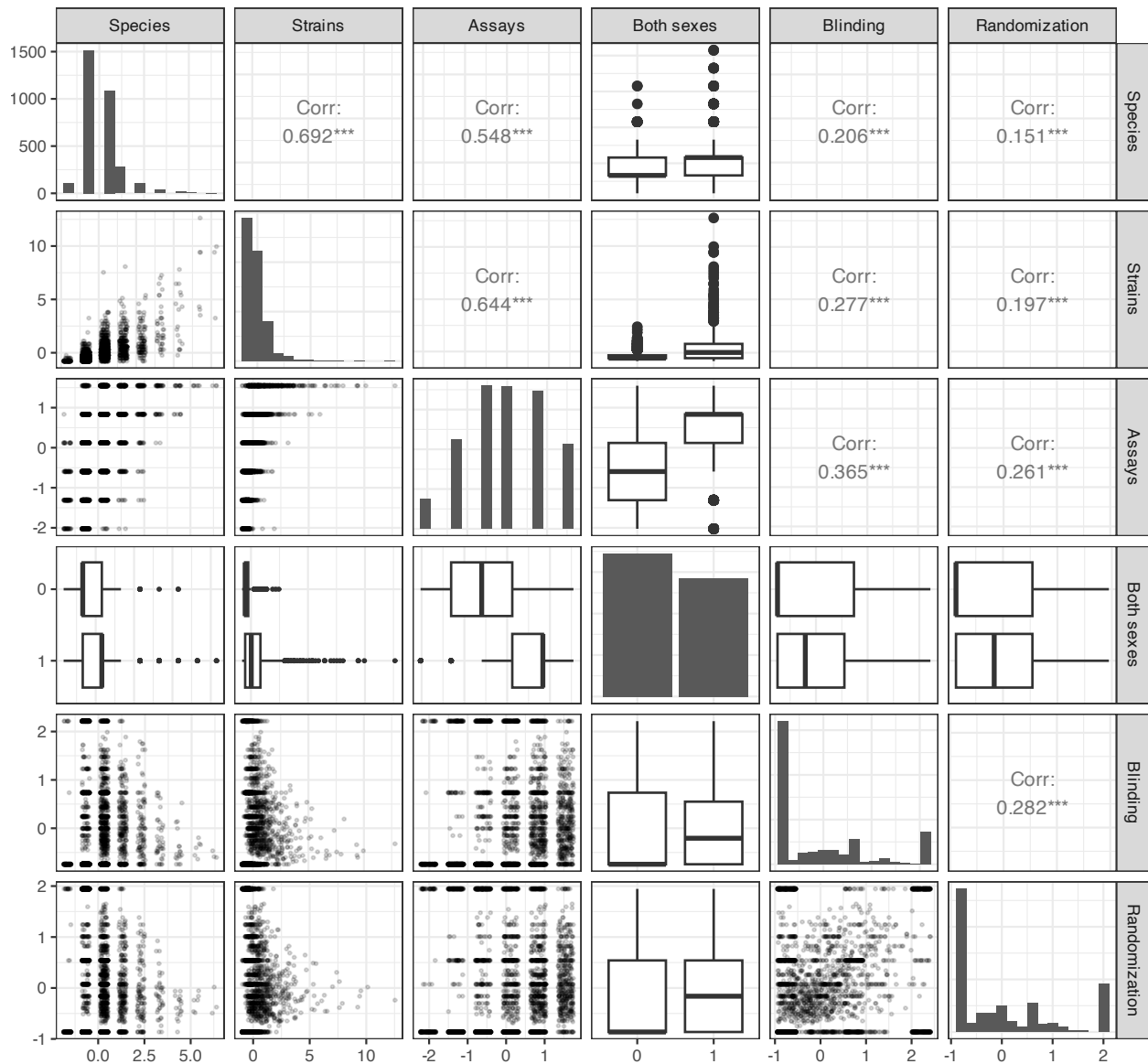


Figure 1: Scatter-plot matrix of the six predictors. Counts are jittered to reduce over-plotting; upper panels give Spearman correlations.

```

ggsave(file.path(viz_dir, "fig_pairs.pdf"), p_pairs, width = 7, height = 6.5) # ... then save

```

3.2 Spearman correlation heatmap (appropriate variables only)

Rank correlation is used to quantify the dependency of the ordered features.

```
cor_mat <- cor(d[, appropriate], method = "spearman")
cor_long <- as.data.frame(as.table(cor_mat)); names(cor_long) <- c("V1", "V2", "rho")

nice <- c(species_unique="Species", strain_unique="Strains", assay_unique="Assays",
          prop_blinding="Blinding", prop_randomization="Randomization")
p_heat <- ggplot(cor_long, aes(V1, V2, fill = rho)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", rho)), size = 3) +
  scale_fill_gradient2(low = "#2166AC", mid = "white", high = "#B2182B",
                      midpoint = 0, limits = c(-1, 1)) +
  scale_x_discrete(labels = nice) + scale_y_discrete(labels = nice) +
  theme(axis.text.x = element_text(angle = 30, hjust = 1)) +
  labs(x = NULL, y = NULL, fill = "Spearman")

p_heat
```

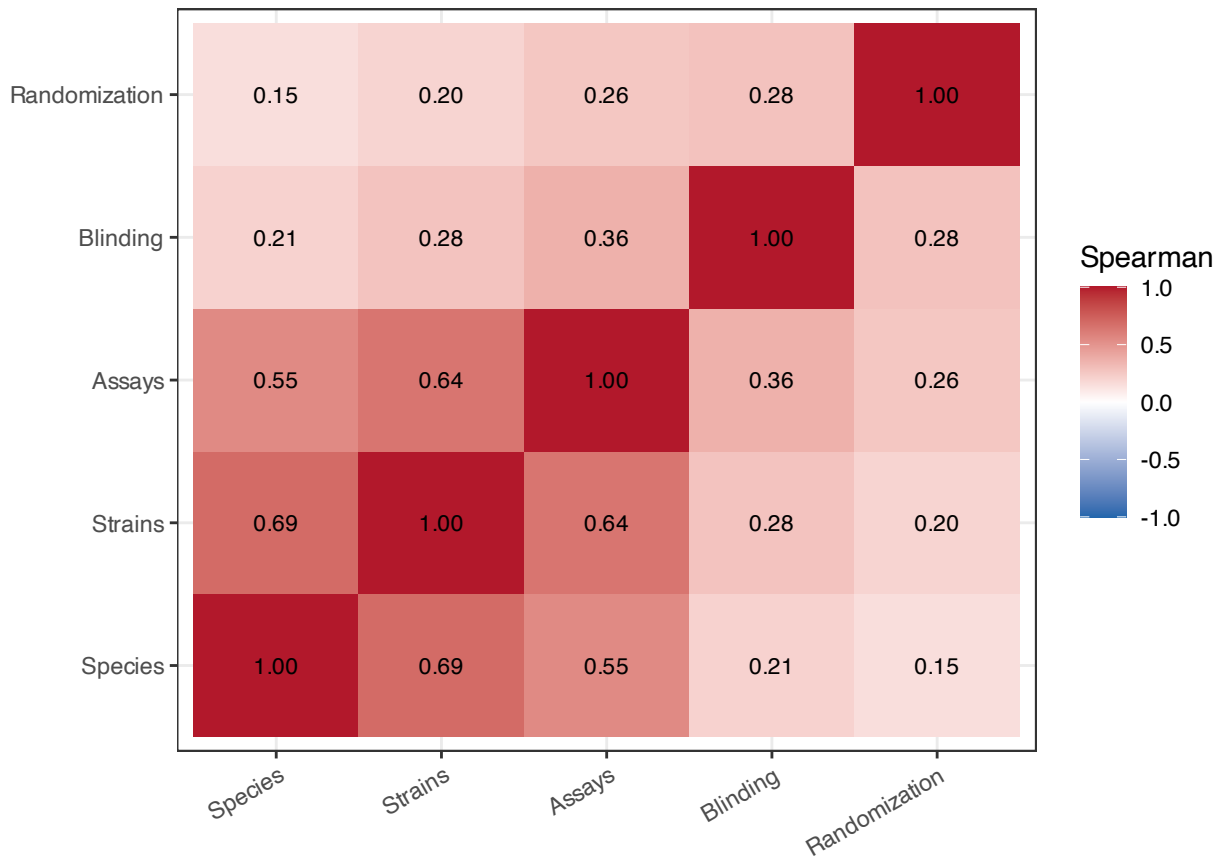


Figure 2: Spearman correlation among the ordinal/continuous predictors. Diversity counts correlate moderately (Species-Strains 0.69); rigor proportions are only weakly related to the rest.

```
ggsave(file.path(viz_dir, "fig_corr_heatmap.pdf"), p_heat, width = 6.4, height = 5.2)
```

```
kable(round(cor_mat, 2), booktabs = TRUE,
      caption = "Spearman correlation (appropriate variables).")
```

Table 1: Spearman correlation (appropriate variables).

	species_unique	strain_unique	assay_unique	prop_blinding	prop_randomization
species_unique	1.00	0.69	0.55	0.21	0.15
strain_unique	0.69	1.00	0.64	0.28	0.20
assay_unique	0.55	0.64	1.00	0.36	0.26
prop_blinding	0.21	0.28	0.36	1.00	0.28
prop_randomization	0.15	0.20	0.26	0.28	1.00

3.3 Collinearity check via variance inflation factor (VIF)

What the VIF measures:

Very high dependency between predictors result in large (inflated) confidence intervals for the coefficients, because it is not clear which variable explains what.

To quantify the VIF of predictor X_j , we regress it on all the other predictors. To quantify the overlap with the other predictors, we read off the regression's R_j^2 . The variance inflation factor is

$$\text{VIF}_j = \frac{1}{1 - R_j^2}.$$

This VIF is the factor by which overlap with the other predictors inflates the *variance* of $\hat{\beta}_j$; Rule of thumb: <5 fine, 5-10 moderate, >10 severe.

```
glm_fit <- glm(target_num ~ species_unique + strain_unique + assay_unique +
               tested_both_sexes_binary + prop_blinding +
               prop_randomization, data = d, family = binomial())
vif_tab <- data.frame(Predictor = lab[names(vif(glm_fit))], VIF = as.numeric(vif(glm_fit)))
kable(vif_tab, digits = 2, row.names = FALSE, booktabs = TRUE,
      caption = "Variance inflation factors. All well below 5 -> no problematic collinearity.")
```

Table 2: Variance inflation factors. All well below 5 -> no problematic collinearity.

Predictor	VIF
Species (count)	1.80
Strains (count)	1.90
Assays (count)	1.66
Both sexes (0/1)	1.26
Blinding (prop.)	1.12
Randomization (prop.)	1.08

Overall: dependency between predictors is measurable but modest, meaning the additive model is not destabilised by collinearity.

4 Reproducing the paper's logistic regression

```
or_tab <- data.frame(
  Predictor = lab[gsub("1$", "", names(coef(glm_fit)))][-1],
  AOR      = exp(coef(glm_fit))[-1],
  CI_low   = exp(confint.default(glm_fit))[-1, 1],
  CI_high  = exp(confint.default(glm_fit))[-1, 2],
  p_value  = summary(glm_fit)$coefficients[-1, 4], row.names = NULL)
kable(or_tab, digits = 3, booktabs = TRUE,
      caption = "Adjusted odds ratios -- reproduces the paper (Species 1.26, Blinding 1.68, ...).")
```

Table 3: Adjusted odds ratios – reproduces the paper (Species 1.26, Blinding 1.68, ...).

Predictor	AOR	CI_low	CI_high	p_value
Species (count)	1.284	1.158	1.425	0.000
Strains (count)	1.082	0.968	1.210	0.166
Assays (count)	0.904	0.824	0.992	0.033
Both sexes (0/1)	0.936	0.796	1.100	0.422
Blinding (prop.)	1.186	1.099	1.280	0.000
Randomization (prop.)	0.973	0.904	1.048	0.474

```
p_forest <- ggplot(or_tab, aes(x = AOR, y = reorder(Predictor, AOR))) +
  geom_vline(xintercept = 1, linetype = "dashed", colour = "grey50") +
  geom_errorbarh(aes(xmin = CI_low, xmax = CI_high), height = 0.2) +
  geom_point(size = 2.4, colour = "#2C7BB6") +
  scale_x_log10() + labs(x = "Adjusted odds ratio (log scale)", y = NULL)
```

p_forest

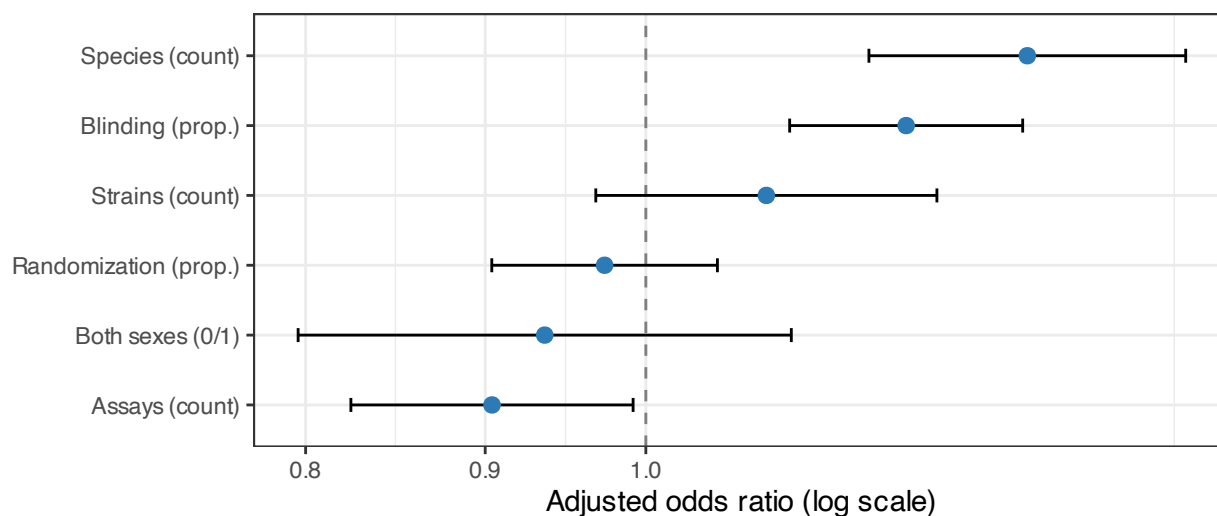


Figure 3: Forest plot of adjusted odds ratios (log scale).

```
ggsave(file.path(viz_dir, "fig_forest.pdf"), p_forest, width = 7, height = 3.2)
```

5 Random forest and model comparison

We now fit a random forest, which is free to model interactions and non-linearities, and ask whether that extra flexibility improves the fit, before checking for modeled interactions or non-linearity.

```
rf <- randomForest(target_f ~ species_unique + strain_unique + assay_unique +
                    tested_both_sexes_binary + prop_blinding +
                    prop_randomization, data = d,
                    ntree = 500, nodesize = 5)
```

To compare the prediction performance of the logistic regression and RF fit we use cross-validated AUC, log-loss and the Brier score.

```
# ---- small metric helpers -----
auc <- function(score, y) { # rank-based AUC (Mann-Whitney)
  r <- rank(score); n1 <- sum(y == 1); n0 <- sum(y == 0)
  (sum(r[y == 1]) - n1 * (n1 + 1) / 2) / (n1 * n0)
}
logloss <- function(p, y) { p <- pmin(pmax(p, 1e-15), 1 - 1e-15)
  -mean(y * log(p) + (1 - y) * log(1 - p)) }
brier <- function(p, y) mean((p - y)^2)

# ---- 10-fold cross-validation for both models -----
K <- 10; folds <- sample(rep(1:K, length.out = nrow(d)))
form <- target_num ~ species_unique + strain_unique + assay_unique +
  tested_both_sexes_binary + prop_blinding + prop_randomization
cv <- lapply(1:K, function(k) {
  tr <- d[folds != k, ]; te <- d[folds == k, ]; yte <- te$target_num
  # logistic regression
  g <- glm(form, data = tr, family = binomial())
  pg <- predict(g, te, type = "response")
  # random forest (positive class = "yes")
  rf_k <- randomForest(target_f ~ species_unique + strain_unique + assay_unique +
    tested_both_sexes_binary + prop_blinding +
    prop_randomization, data = tr, ntree = 500, nodesize = 5)
  pr <- predict(rf_k, te, type = "prob")[, "yes"]
  data.frame(
    Logistic_AUC = auc(pg, yte), RF_AUC = auc(pr, yte),
    Logistic_LL = logloss(pg, yte), RF_LL = logloss(pr, yte),
    Logistic_Brier = brier(pg, yte), RF_Brier = brier(pr, yte))
})
cv <- bind_rows(cv)

cmp <- data.frame(
  Metric = c("AUC (higher better)", "Log-loss (lower better)", "Brier (lower better)"),
  Logistic = c(mean(cv$Logistic_AUC), mean(cv$Logistic_LL), mean(cv$Logistic_Brier)),
  RandomForest = c(mean(cv$RF_AUC), mean(cv$RF_LL), mean(cv$RF_Brier)))
kable(cmp, digits = 3, booktabs = TRUE,
      caption = "10-fold cross-validated performance. The two models are essentially tied.")
```

Table 4: 10-fold cross-validated performance. The two models are essentially tied.

Metric	Logistic	RandomForest
AUC (higher better)	0.571	0.559

Metric	Logistic	RandomForest
Log-loss (lower better)	0.671	0.797
Brier (lower better)	0.239	0.268

The forest does not beat the additive logistic model on any prediction metric. That already suggests the additive form is adequate. The ICE plots below show *why*.

6 Individual dependency (ICE) plots

6.1 What an ICE curve is (in plain terms)

We pick one predictor (e.g. number of species) for which we want to investigate its effect on the probability of translation.

Take one observation, here one drug-disease pair and make a prediction with the fitted model and plot this as red point in a plot translation probability vs predictor value. Then keep all features fixed except number of species. Slide the feature (number of species) across its whole range, and make for each value a new prediction for the probability of translation. Join those predictions into a line: that line is the pair's **ICE curve**. One curve per pair, one panel per feature. We pick a random subset of observations to avoid overplotting.

Reading the curves:

- **Rising / falling** -> the feature increases / decreases the probability.
- Curves **parallel** -> same effect for every observation = no interaction; curves **crossing / fanning out** -> effect depends on other features = interaction.
- **Green** = the mean of all curves (the partial-dependence curve, i.e. the average effect). **Red dot** = the predicted value if the feature is set to the observed value (not extrapolated).
- **Rug** = where the real data lie; curves away from the ticks are extrapolation.

6.2 Logistic regression vs. random forest, side by side

```
ice_feats <- appropriate                                # 5 ordinal/continuous features
sample_idx <- sample(nrow(d), 50)                      # shared random subset -> avoid crowding

make_grid <- function(x, f)                             # values to slide the feature over
  if (grepl("^prop_", f)) seq(0, 1, length.out = 40) else seq(min(x), max(x), by = 1)

# build all ICE curves for one prediction function
build_ice <- function(predict_prob, model_name) {
  do.call(rbind, lapply(ice_feats, function(f) {
    g <- make_grid(d[[f]], f)
    do.call(rbind, lapply(sample_idx, function(i) {
      nd <- d[rep(i, length(g)), ]; nd[[f]] <- g
      data.frame(model = model_name, feature = f, row_id = i,
                  x = g, prob = predict_prob(nd))
    })))
  })))
}

# the single observed (non-extrapolated) point per curve
build_pts <- function(predict_prob, model_name)
  do.call(rbind, lapply(ice_feats, function(f)
    data.frame(model = model_name, feature = f, x = d[[f]][sample_idx],
               prob = predict_prob(d[sample_idx, ]))))
```



```

predict_logit <- function(nd) as.numeric(predict(glm_fit, nd, type = "response"))
predict_rf    <- function(nd) predict(rf, nd, type = "prob")[, "yes"]

curves <- rbind(build_ice(predict_logit, "Logistic regression"),
                build_ice(predict_rf,    "Random forest"))
points_ <- rbind(build_pts(predict_logit, "Logistic regression"),
                 build_pts(predict_rf,    "Random forest"))

# green mean curve = average of the ICE curves = partial-dependence (PD) curve
pd_curve <- curves %>%
  group_by(model, feature, x) %>%
  summarise(prob = mean(prob), .groups = "drop")

# one row per feature (left = logistic, right = RF), stacked with patchwork
panel <- function(f) {
  ggplot() +
    geom_line(data = subset(curves, feature == f),
              aes(x, prob, group = row_id), colour = "steelblue",
              alpha = 0.25, linewidth = 0.35) +
    geom_point(data = subset(points_, feature == f),
               aes(x, prob), colour = "red", size = 0.8, alpha = 0.8) +
    geom_line(data = subset(pd_curve, feature == f),
              aes(x, prob), colour = "darkgreen", linewidth = 1.1) +
    geom_rug(data = data.frame(x = d[[f]]), aes(x = x), sides = "b", alpha = 0.10) +
    facet_wrap(~ model, ncol = 2) +
    coord_cartesian(ylim = c(0, 1)) +
    labs(title = lab[f], x = NULL, y = "P(translated)")
}
p_ice <- wrap_plots(lapply(ice_feats, panel), ncol = 1)

p_ice

```

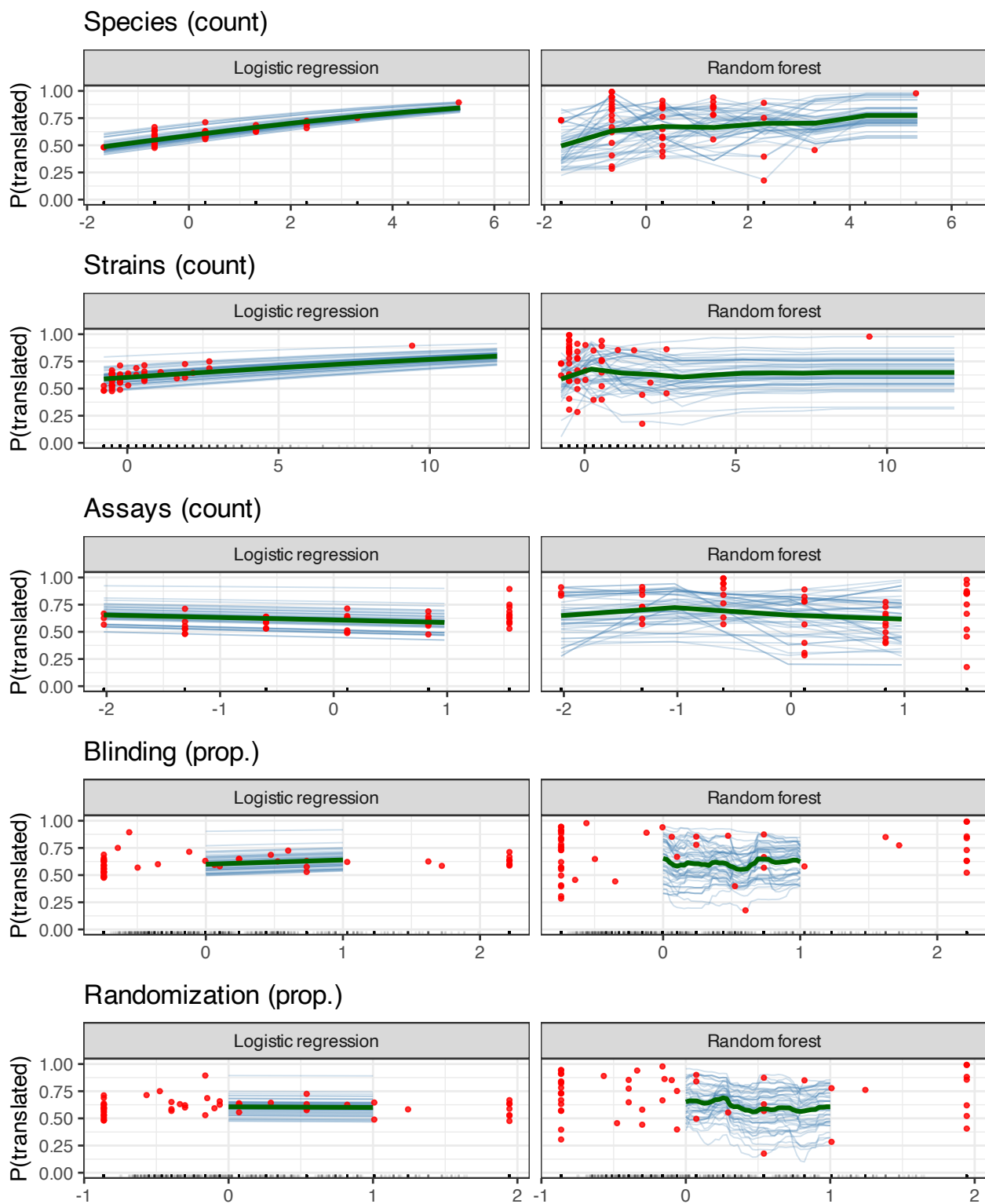


Figure 4: ICE curves on the SAME 50 random pairs. Left: logistic regression; right: random forest. Blue = one pair; green = mean curve (partial dependence); red = observed value (not extrapolated); ticks = rug.

```
ggsave(file.path(viz_dir, "fig_ICE_logreg_vs_rf.pdf"), p_ice, width = 7, height = 8.5)
```

6.3 Interpreting the ICE plots

- **Same directions.** In both columns *Species* and *Blinding* rise, *Assays* tilts down slightly, *Strains* rises gently, *Randomization* is flat – matching the odds ratios (Species 1.26, Blinding 1.68 > 1; Assays 0.96, Randomization 0.95 near/below 1). The forest recovers the same story, and its green mean curve tracks the logistic one.
- **Curves stay parallel.** Left panel: Parallel curves by construction (additive logistic regression which does not allow for interaction) ; Right panel: also roughly parallel curves, meaning the forest finds no interaction the additive model misses.
- **No strong non-linearities.** The forest curves are slightly wigglier and flatten in the sparse tails (read against the rug), but show no consistent bends the additive model would miss.

7 Conclusion: is the additive logistic model justified?

Yes. Three independent lines of evidence agree:

1. **Modest dependency.** Predictors correlate only moderately and every VIF is below 2 – no redundancy, stable coefficients.
2. **No predictive gain from flexibility.** A random forest free to exploit interactions and non-linearities matches the additive model on cross-validated AUC, log-loss and Brier – it finds nothing extra to exploit.
3. **Parallel ICE curves.** The forest’s dependency curves stay parallel and monotone, mirroring the logistic coefficients, so there is no hidden interaction or curvature.

Adding interaction or non-linear terms is therefore not warranted; it would add variance and hurt interpretability. Reporting adjusted, per-predictor associations under an additive logistic model is an appropriate, defensible choice.

8 Suggested response to Reviewer #4 (point 1)

We thank the reviewer and examined predictor interdependence and the adequacy of the additive form empirically. The predictors show only modest mutual dependency (largest Spearman $\rho = 0.69$, between species and strains) and all variance inflation factors are below 2, indicating no problematic collinearity. To probe for hidden interactions or non-linearities, we fitted a random forest, which is free to exploit both; it did not outperform the additive logistic model in 10-fold cross-validation (AUC 0.57 vs. 0.58; log-loss and Brier essentially identical), and its individual conditional expectation curves were largely parallel and monotone, mirroring the logistic coefficients. We therefore conclude that the additive specification adequately captures the structure and have added these analyses to the supplement. We will add a corresponding sentence in the manuscript.

```
## R version 4.3.3 (2024-02-29)
## Platform: aarch64-apple-darwin20 (64-bit)
## Running under: macOS Sonoma 14.3.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
##
## time zone: Europe/Zurich
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] knitr_1.45      car_3.1-3      carData_3.0-5
## [4] randomForest_4.7-1.2 patchwork_1.3.0 GGally_2.2.1
## [7] ggplot2_3.5.2   tidyr_1.3.1    dplyr_1.1.4
## [10] readr_2.1.5
##
## loaded via a namespace (and not attached):
## [1] utf8_1.2.4      generics_0.1.3  hms_1.1.3       digest_0.6.37
## [5] magrittr_2.0.3  evaluate_1.0.4  grid_4.3.3      RColorBrewer_1.1-3
## [9] fastmap_1.2.0   plyr_1.8.9      Formula_1.2-5    purrr_1.0.2
## [13] fansi_1.0.6     scales_1.4.0    textshaping_1.0.0 abind_1.4-8
## [17] cli_3.6.5       rlang_1.1.6     crayon_1.5.3     bit64_4.6.0-1
## [21] withr_3.0.2     yaml_2.3.10     tools_4.3.3      parallel_4.3.3
## [25] tzdb_0.5.0      ggstats_0.10.0  vctrs_0.6.5      R6_2.6.1
## [29] lifecycle_1.0.4 bit_4.5.0.1     vroom_1.6.5      ragg_1.4.0
## [33] pkgconfig_2.0.3 pillar_1.9.0    gtable_0.3.6     glue_1.8.0
## [37] Rcpp_1.0.13     systemfonts_1.1.0 xfun_0.42        tibble_3.2.1
## [41] tidyselect_1.2.1 rstudioapi_0.15.0 farver_2.1.1     htmltools_0.5.8.1
## [45] rmarkdown_2.29  labeling_0.4.3  compiler_4.3.3
```